ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH

TRƯỜNG ĐẠI HỌC BÁCH KHOA

KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Giảng viên hướng dẫn:

Lớp:

TP. Hồ Chí Minh, tháng 10/2026

Đánh giá thành viên

STT	Thành viên nhóm	MSSV	Đánh giá
1			100%
2			100%
3			100%
4			100%

Mục lục

1 Lab 1 **4**
1.1 Exercise 1 - Lab 1 . . . 4
1.2 Exercise 2 - Lab 1 . . . 4
1.3 Exercise 3 - Lab 1 . . . 4
1.4 Exercise 4 - Lab 1 . . . 4
1.5 Exercise 5 - Lab 1 . . . 4
1.6 Exercise 6 - Lab 1 . . . 4
1.7 Trình bày schematic . . . 4
1.8 Exercise 7 - Lab 1 . . . 4
1.8.1 Đoạn chương trình . . . 4
1.9 Exercise 8 - Lab 1 . . . 4
1.10 Exercise 9 - Lab 1 . . . 5
1.11 Exercise 10 - Lab 1 . . . 5

2 Lab 2 **8**

3 Exercise 6 - Lab 2 **8**

4 Exercise 6 - Lab 2 **9**

5 Exercise 6 - Lab 2 **11**

6 Exercise 6 - Lab 2 **12**

7 Exercise 6 - Lab 2 **14**

8 Exercise 6 - Lab 2 **16**

9 Exercise 7 - Lab 2 **17**

10 Exercise 8 - Lab 2 **19**

11 Exercise 6 - Lab 2 **20**

12 Exercise 6 - Lab 2 **22**

13 Lab 3 **25**
13.1 Exercise 1 - Lab 3 . . . 25
13.2 Exercise 3 - Lab 3 . . . 26
13.3 Exercise 4 - Lab 3 . . . 28

13.4 Exercise 5 - Lab 3: Button Debouncing Abstraction Layer 30

14 Exercise 5 - Lab 3: Button Debouncing Abstraction Layer 32

14.1 Mã nguồn Cài đặt 33

1 Lab 1

1.1 Exercise 1 - Lab 1

1.2 Exercise 2 - Lab 1

1.3 Exercise 3 - Lab 1

1.4 Exercise 4 - Lab 1

1.5 Exercise 5 - Lab 1

1.6 Exercise 6 - Lab 1

Mục tiêu:

- Trình bày schematic
- Triển khai 1 chương trình đơn giản để kiểm tra kết nối từng đèn LED

Yêu cầu: Chương trình này cần bật đèn LED một cách có trình tự Giải thích:

1.7 Trình bày schematic

Hình vẽ sau mô tả mạch schematic

Phân tích : Cổng led 1 nối với

Đoạn mã sau hiển thị 7SEG function:

1.8 Exercise 7 - Lab 1

Yêu cầu: Hiện thực hàm clearAllClock() để tắt tất cả 12 LED
Ý tưởng:

1.8.1 Đoạn chương trình

Đoạn mã sau hiện thực hàm clearAllClock:

1.9 Exercise 8 - Lab 1

Yêu cầu: Hiện thực hàm setNumberOnClock(int num). Giá trị đầu vào cho hàm này nằm trong khoảng từ 0 đến 11, và đèn LED tương ứng sẽ bật.

Ý tưởng: Hàm nhận tham số đầu vào là vị trí của LED (từ 0 đến 11) và sử dụng mảng `pins` để xác định chân GPIO cần xuất tín hiệu (`GPIO_PIN_SET`).

```
void setNumberOnClock(int num) {
    if(num >= 0 && num <= 11) {
        HAL_GPIO_WritePin(GPIOA, pins[num], GPIO_PIN_SET);
    }
}
```

Listing 1: Hàm setNumberOnClock

1.10 Exercise 9 - Lab 1

Yêu cầu: Hiện thực hàm clearNumberOnClock(int num). Giá trị đầu vào cho hàm này nằm trong khoảng từ 0 đến 11, và đèn LED tương ứng sẽ tắt.

Ý tưởng: Hàm nhận tham số đầu vào là vị trí của LED (từ 0 đến 11) và sử dụng mảng `pins` để xác định chân GPIO cần xuất tín hiệu tắt `GPIO_PIN_RESET`.

```
void clearNumberOnClock(int num) {
    if(num >= 0 && num <= 11) {
        HAL_GPIO_WritePin(GPIOA, pins[num], GPIO_PIN_RESET);
    }
}
```

Listing 2: Hàm clearNumberOnClock

1.11 Exercise 10 - Lab 1

Yêu cầu:Tích hợp toàn bộ các hàm đã xây dựng từ những bài tập trước để tạo thành một hệ thống đồng hồ sử dụng 12 đèn LED[cite: 30]. Tại bất kỳ thời điểm nào, chỉ có 3 đèn LED được phép bật sáng để thể hiện lần lượt 3 thông tin: giờ, phút và giây[cite: 30].

Ý tưởng: 12 bóng LED này được kết nối trực tiếp với 12 chân GPIO liên tiếp của vi điều khiển STM32, bắt đầu từ chân `PA4` đến `PA15`[cite: 29]. Nhóm chọn chân `PA15` tương ứng với vị trí 12 giờ, `PA4` là 1 giờ. Để điều khiển đồng hồ, hệ thống sử dụng lại các hàm phụ trợ đã được yêu cầu ở các bài tập 7, 8 và 9[cite: 30]. Do phút và giây là từ 0 đến 59, trong khi mặt đồng hồ chỉ có 12 vị trí (0 đến 11), hệ thống chia lấy phần nguyên của giá trị phút và giây cho 5 để ánh xạ vào đúng vị trí của đèn LED tương ứng. Nhịp đếm thời gian 1 giây được tạo ra thông qua hàm `HAL_Delay(1000)`. Trong trường hợp giá trị của kim giây, kim phút và kim giờ trùng nhau tại một vị trí, chân GPIO tương ứng của vị trí đó vẫn sẽ duy trì trạng thái mức cao (bật sáng) bình thường.

```

int hour = 0;
int minute = 0;
int second = 0;
void clearAllClock() {
```

```
    for(int i = 0; i < 12; i++) {
        HAL_GPIO_WritePin(GPIOA, pins[i], GPIO_PIN_RESET);
    }
}
void setNumberOnClock(int num) {
    if(num >= 0 && num <= 11) {
        HAL_GPIO_WritePin(GPIOA, pins[num], GPIO_PIN_SET);
    }
}
void clearNumberOnClock(int num) {
    if(num >= 0 && num <= 11) {
        HAL_GPIO_WritePin(GPIOA, pins[num], GPIO_PIN_RESET);
    }
}

/* USER CODE BEGIN WHILE */
  while (1)
  {
        clearAllClock();


        int second_pos = second / 5;
        int minute_pos = minute / 5;
        int hour_pos = hour;

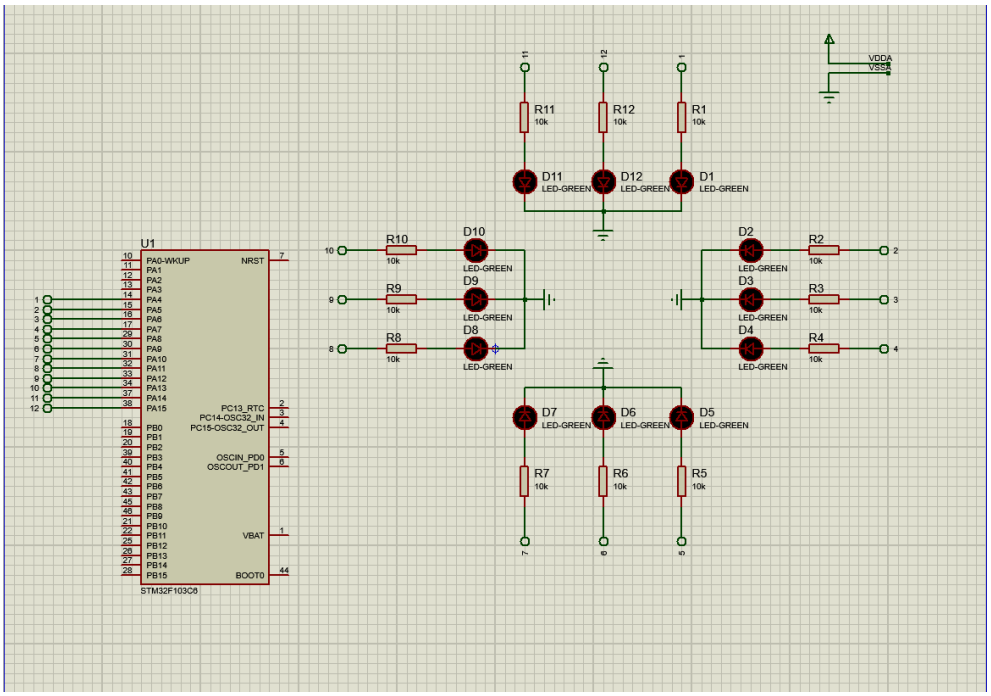

        setNumberOnClock(second_pos);
        setNumberOnClock(minute_pos);
        setNumberOnClock(hour_pos);


        HAL_Delay(10);


        second++;
        if(second >= 60) {
            second = 0;
            minute++;
        }
        if(minute >= 60) {
            minute = 0;
            hour++;
        }
        if(hour >= 12) {
            hour = 0;
```

```
51          }
52  /* USER CODE END WHILE */
53  }
```

Listing 3: Tích hợp



Hình 1: *Mô phỏng mạch tích hợp*

2 Lab 2

3 Exercise 6 - Lab 2

Yêu cầu: Mục tiêu chính là giảm thiểu độ phức tạp bên trong hàm ngắt timer để tránh lỗi ngắt lồng nhau gây treo hệ thống. Tạo ra một software timer đếm lùi sau mỗi lần ngắt timer xảy ra mỗi 10ms. Sử dụng timer phần mềm này để thay thế và loại bỏ hàm `HAL_Delay(1000)` trong hàm `main()`, nhằm tạo ra một hệ thống xử lý trễ không đồng bộ.

Ý tưởng: Khai báo các biến toàn cục để quản lý timer phần mềm gồm: `timer_counter`, `timer_flag` và `TIMER_CYCLE`. Hiện thực hàm `setTimer(int duration)` để nạp giá trị đếm bằng cách lấy thời gian mong muốn chia cho chu kỳ. Hàm `timer_run()` để lùi biến `timer_counter` lớn hơn 0, khi biến đếm về 0 thì cờ bật lên 1.

```
int timer_counter = 0;
int timer_flag = 0;
int TIMER_CYCLE = 10;

void setTimer(int duration){
    timer_counter = duration / TIMER_CYCLE;
    timer_flag = 0;
}

void timer_run(){
    if(timer_counter > 0){
        timer_counter--;
        if(timer_counter == 0) timer_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if(htim->Instance == TIM2){
        timer_run();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|
   GPIO_PIN_9, GPIO_PIN_SET);
        update7SEG(index_led);

        if(second_counter == 2 || second_counter == 4) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        }

        second++;
        if(second >= 60) {
```

```
            second = 0;
            minute++;
        }

        if(minute >= 60) {
            minute = 0;
            hour++;
        }

        if(hour >= 24) {
            hour = 0;
        }

        updateClockBuffer();
    }
}
```

Listing 4: Phần code hiện thực

- **Report 1:** Nếu xóa dòng `setTimer(1000);` trước vòng lặp while, biến `timer_counter` ban đầu sẽ bằng 0. Cờ sẽ không bao giờ được bật lên 1, dẫn đến LED không bao giờ nhấp nháy.
- **Report 2:** Nếu đổi thành `setTimer(1);`, thời gian đếm lùi sẽ là $1/10 = 0$. LED sẽ không nhấp nháy.
- **Report 3:** Nếu đổi thành `setTimer(10);`, LED sẽ nhấp nháy mỗi 10ms. LED sẽ nhấp nháy nhanh, chúng ta vẫn sẽ không nhìn ra được sự nhấp nháy.

4 Exercise 6 - Lab 2

Yêu cầu: Mục tiêu chính là giảm thiểu độ phức tạp bên trong hàm ngắt timer để tránh lỗi ngắt lồng nhau gây treo hệ thống. Tạo ra một software timer đếm lùi sau mỗi lần ngắt timer xảy ra mỗi 10ms. Sử dụng timer phần mềm này để thay thế và loại bỏ hàm `HAL_Delay(1000)` trong hàm `main()`, nhằm tạo ra một hệ thống xử lý trễ không đồng bộ.

Ý tưởng: Khai báo các biến toàn cục để quản lý timer phần mềm gồm: `timer_counter`, `timer_flag` và `TIMER_CYCLE`. Hiện thực hàm `setTimer(int duration)` để nạp giá trị đếm bằng cách lấy thời gian mong muốn chia cho chu kỳ. Hàm `timer_run()` để lùi biến `timer_counter` lớn hơn 0, khi biến đếm về 0 thì cờ bật lên 1.

```
int timer_counter = 0;
int timer_flag = 0;
```

```
int TIMER_CYCLE = 10;

void setTimer(int duration){
    timer_counter = duration / TIMER_CYCLE;
    timer_flag = 0;
}

void timer_run(){
    if(timer_counter > 0){
        timer_counter--;
        if(timer_counter == 0) timer_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if(htim->Instance == TIM2){
        timer_run();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|
   GPIO_PIN_9, GPIO_PIN_SET);
        update7SEG(index_led);

        if(second_counter == 2 || second_counter == 4) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        }

        second++;
        if(second >= 60) {
            second = 0;
            minute++;
        }

        if(minute >= 60) {
            minute = 0;
            hour++;
        }

        if(hour >= 24) {
            hour = 0;
        }

        updateClockBuffer();
    }
}
```

Listing 5: Phần code hiện thực

- **Report 1:** Nếu xóa dòng `setTimer(1000);` trước vòng lặp while, biến `timer_counter` ban đầu sẽ bằng 0. Cờ sẽ không bao giờ được bật lên 1, dẫn đến LED không bao giờ nhấp nháy.
- **Report 2:** Nếu đổi thành `setTimer(1);`, thời gian đếm lùi sẽ là $1/10 = 0$. LED sẽ không nhấp nháy.
- **Report 3:** Nếu đổi thành `setTimer(10);`, LED sẽ nhấp nháy mỗi 10ms. LED sẽ nhấp nháy nhanh, chúng ta vẫn sẽ không nhìn ra được sự nhấp nháy.

5 Exercise 6 - Lab 2

Yêu cầu: Mục tiêu chính là giảm thiểu độ phức tạp bên trong hàm ngắt timer để tránh lỗi ngắt lồng nhau gây treo hệ thống. Tạo ra một software timer đếm lùi sau mỗi lần ngắt timer xảy ra mỗi 10ms. Sử dụng timer phần mềm này để thay thế và loại bỏ hàm `HAL_Delay(1000)` trong hàm `main()`, nhằm tạo ra một hệ thống xử lý trễ không đồng bộ.

Ý tưởng: Khai báo các biến toàn cục để quản lý timer phần mềm gồm: `timer_counter`, `timer_flag` và `TIMER_CYCLE`. Hiện thực hàm `setTimer(int duration)` để nạp giá trị đếm bằng cách lấy thời gian mong muốn chia cho chu kỳ. Hàm `timer_run()` để lùi biến `timer_counter` lớn hơn 0, khi biến đếm về 0 thì cờ bật lên 1.

```
int timer_counter = 0;
int timer_flag = 0;
int TIMER_CYCLE = 10;

void setTimer(int duration){
    timer_counter = duration / TIMER_CYCLE;
    timer_flag = 0;
}

void timer_run(){
    if(timer_counter > 0){
        timer_counter--;
        if(timer_counter == 0) timer_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if(htim->Instance == TIM2){
        timer_run();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|
   GPIO_PIN_9, GPIO_PIN_SET);
```

```
        update7SEG(index_led);

        if(second_counter == 2 || second_counter == 4) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        }

        second++;
        if(second >= 60) {
            second = 0;
            minute++;
        }

        if(minute >= 60) {
            minute = 0;
            hour++;
        }

        if(hour >= 24) {
            hour = 0;
        }

        updateClockBuffer();
    }
}
```

Listing 6: Phần code hiện thực

- **Report 1:** Nếu xóa dòng `setTimer(1000);` trước vòng lặp while, biến `timer_counter` ban đầu sẽ bằng 0. Cờ sẽ không bao giờ được bật lên 1, dẫn đến LED không bao giờ nhấp nháy.
- **Report 2:** Nếu đổi thành `setTimer(1);`, thời gian đếm lùi sẽ là $1/10 = 0$. LED sẽ không nhấp nháy.
- **Report 3:** Nếu đổi thành `setTimer(10);`, LED sẽ nhấp nháy mỗi 10ms. LED sẽ nhấp nháy nhanh, chúng ta vẫn sẽ không nhìn ra được sự nhấp nháy.

6 Exercise 6 - Lab 2

Yêu cầu: Mục tiêu chính là giảm thiểu độ phức tạp bên trong hàm ngắt timer để tránh lỗi ngắt lồng nhau gây treo hệ thống. Tạo ra một software timer đếm lùi sau mỗi lần ngắt timer xảy ra mỗi 10ms. Sử dụng timer phần mềm này để thay thế và loại bỏ hàm

`HAL_Delay(1000)` trong hàm `main()`, nhằm tạo ra một hệ thống xử lý trễ không đồng bộ.

Ý tưởng: Khai báo các biến toàn cục để quản lý timer phần mềm gồm: `timer_counter`, `timer_flag` và `TIMER_CYCLE`. Hiện thực hàm `setTimer(int duration)` để nạp giá trị đếm bằng cách lấy thời gian mong muốn chia cho chu kỳ. Hàm `timer_run()` để lùi biến `timer_counter` lớn hơn 0, khi biến đếm về 0 thì cờ bật lên 1.

```
int timer_counter = 0;
int timer_flag = 0;
int TIMER_CYCLE = 10;

void setTimer(int duration){
    timer_counter = duration / TIMER_CYCLE;
    timer_flag = 0;
}

void timer_run(){
    if(timer_counter > 0){
        timer_counter--;
        if(timer_counter == 0) timer_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if(htim->Instance == TIM2){
        timer_run();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|
   GPIO_PIN_9, GPIO_PIN_SET);
        update7SEG(index_led);

        if(second_counter == 2 || second_counter == 4) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        }

        second++;
        if(second >= 60) {
            second = 0;
            minute++;
        }

        if(minute >= 60) {
            minute = 0;
            hour++;
        }
```

```
37
38          if(hour >= 24) {
39              hour = 0;
40          }
41
42          updateClockBuffer();
43      }
44  }
```

Listing 7: Phần code hiện thực

- **Report 1:** Nếu xóa dòng `setTimer(1000);` trước vòng lặp while, biến `timer_counter` ban đầu sẽ bằng 0. Cờ sẽ không bao giờ được bật lên 1, dẫn đến LED không bao giờ nhấp nháy.
- **Report 2:** Nếu đổi thành `setTimer(1);`, thời gian đếm lùi sẽ là $1/10 = 0$. LED sẽ không nhấp nháy.
- **Report 3:** Nếu đổi thành `setTimer(10);`, LED sẽ nhấp nháy mỗi 10ms. LED sẽ nhấp nháy nhanh, chúng ta vẫn sẽ không nhìn ra được sự nhấp nháy.

7 Exercise 6 - Lab 2

Yêu cầu: Mục tiêu chính là giảm thiểu độ phức tạp bên trong hàm ngắt timer để tránh lỗi ngắt lồng nhau gây treo hệ thống. Tạo ra một software timer đếm lùi sau mỗi lần ngắt timer xảy ra mỗi 10ms. Sử dụng timer phần mềm này để thay thế và loại bỏ hàm `HAL_Delay(1000)` trong hàm `main()`, nhằm tạo ra một hệ thống xử lý trễ không đồng bộ.

Ý tưởng: Khai báo các biến toàn cục để quản lý timer phần mềm gồm: `timer_counter`, `timer_flag` và `TIMER_CYCLE`. Hiện thực hàm `setTimer(int duration)` để nạp giá trị đếm bằng cách lấy thời gian mong muốn chia cho chu kỳ. Hàm `timer_run()` để lùi biến `timer_counter` lớn hơn 0, khi biến đếm về 0 thì cờ bật lên 1.

```
1  int timer_counter = 0;
2  int timer_flag = 0;
3  int TIMER_CYCLE = 10;
4
5  void setTimer(int duration){
6      timer_counter = duration / TIMER_CYCLE;
7      timer_flag = 0;
8  }
9
10 void timer_run(){
```

```
    if(timer_counter > 0){
        timer_counter--;
        if(timer_counter == 0) timer_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if(htim->Instance == TIM2){
        timer_run();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|
   GPIO_PIN_9, GPIO_PIN_SET);
        update7SEG(index_led);

        if(second_counter == 2 || second_counter == 4) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        }

        second++;
        if(second >= 60) {
            second = 0;
            minute++;
        }

        if(minute >= 60) {
            minute = 0;
            hour++;
        }

        if(hour >= 24) {
            hour = 0;
        }

        updateClockBuffer();
    }
}
```

Listing 8: Phần code hiện thực

- **Report 1:** Nếu xóa dòng `setTimer(1000);` trước vòng lặp while, biến `timer_counter` ban đầu sẽ bằng 0. Cờ sẽ không bao giờ được bật lên 1, dẫn đến LED không bao giờ nhấp nháy.
- **Report 2:** Nếu đổi thành `setTimer(1);`, thời gian đếm lùi sẽ là $1/10 = 0$. LED sẽ không nhấp nháy.

- **Report 3:** Nếu đổi thành `setTimer(10);`, LED sẽ nhấp nháy mỗi 10ms. LED sẽ nhấp nháy nhanh, chúng ta vẫn sẽ không nhìn ra được sự nhấp nháy.

8 Exercise 6 - Lab 2

Yêu cầu: Mục tiêu chính là giảm thiểu độ phức tạp bên trong hàm ngắt timer để tránh lỗi ngắt lồng nhau gây treo hệ thống. Tạo ra một software timer đếm lùi sau mỗi lần ngắt timer xảy ra mỗi 10ms. Sử dụng timer phần mềm này để thay thế và loại bỏ hàm `HAL_Delay(1000)` trong hàm `main()`, nhằm tạo ra một hệ thống xử lý trễ không đồng bộ.

Ý tưởng: Khai báo các biến toàn cục để quản lý timer phần mềm gồm: `timer_counter`, `timer_flag` và `TIMER_CYCLE`. Hiện thực hàm `setTimer(int duration)` để nạp giá trị đếm bằng cách lấy thời gian mong muốn chia cho chu kỳ. Hàm `timer_run()` để lùi biến `timer_counter` lớn hơn 0, khi biến đếm về 0 thì cờ bật lên 1.

```
int timer_counter = 0;
int timer_flag = 0;
int TIMER_CYCLE = 10;

void setTimer(int duration){
    timer_counter = duration / TIMER_CYCLE;
    timer_flag = 0;
}

void timer_run(){
    if(timer_counter > 0){
        timer_counter--;
        if(timer_counter == 0) timer_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if(htim->Instance == TIM2){
        timer_run();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|
   GPIO_PIN_9, GPIO_PIN_SET);
        update7SEG(index_led);

        if(second_counter == 2 || second_counter == 4) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        }

        second++;
```

```
28          if(second >= 60) {
29              second = 0;
30              minute++;
31          }
32
33          if(minute >= 60) {
34              minute = 0;
35              hour++;
36          }
37
38          if(hour >= 24) {
39              hour = 0;
40          }
41
42          updateClockBuffer();
43      }
44  }
```

Listing 9: Phần code hiện thực

- **Report 1:** Nếu xóa dòng `setTimer(1000);` trước vòng lặp while, biến `timer_counter` ban đầu sẽ bằng 0. Cờ sẽ không bao giờ được bật lên 1, dẫn đến LED không bao giờ nhấp nháy.
- **Report 2:** Nếu đổi thành `setTimer(1);`, thời gian đếm lùi sẽ là $1/10 = 0$. LED sẽ không nhấp nháy.
- **Report 3:** Nếu đổi thành `setTimer(10);`, LED sẽ nhấp nháy mỗi 10ms. LED sẽ nhấp nháy nhanh, chúng ta vẫn sẽ không nhìn ra được sự nhấp nháy.

9 Exercise 7 - Lab 2

Yêu cầu: Nâng cấp mã nguồn của Bài tập 5 (đồng hồ số cập nhật giờ, phút, giây) bằng cách áp dụng timer phần mềm và xóa bỏ hàm `HAL_Delay()`. Chuyển tác vụ nhấp nháy đèn DOT vào trong hàm `main()` thay vì để trong ngắt.

Ý tưởng: Mở rộng timer phần mềm thành mảng để nhiều tác vụ đồng thời. Cấp phát timer 0 cho đèn DOT và timer 1 cho đèn LED. Bên trong vòng lặp while, ta kiểm tra cờ của từng timer một cách độc lập. Khi cờ bật, ta xử lý logic tương ứng và cài đặt lại timer đó.

```
1     setTimer(0, 500);  // DOT (500ms)
2     setTimer(1, 1000); //  Clock & LED_RED (1000ms)
3     HAL_TIM_Base_Start_IT(&htim2);
```

```
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
    while (1)
    {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

        if(timer_flag[0] == 1){
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
            setTimer(0, 500);
        }
        if(timer_flag[1] == 1){
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);

            second++;
            if(second >= 60) {
                second = 0;
                minute++;
            }
            if(minute >= 60) {
                minute = 0;
                hour++;
            }
            if(hour >= 24) {
                hour = 0;
            }
            updateClockBuffer();
            setTimer(1, 1000);
        }
    }
  /* USER CODE END 3 */
    void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
      if(htim->Instance == TIM2){
        timer_run();
        HAL_GPIO_WritePin(GPIOA,GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8
                                        |GPIO_PIN_9, GPIO_PIN_SET);
        update7SEG(index_led);
  }

}
```

Listing 10: Phần code hiện thực

10 Exercise 8 - Lab 2

Yêu cầu: Di chuyển hàm quét LED `update7SEG()` từ hàm ngắt timer ra ngoài hàm `()`. Đảm bảo ngắt timer lúc này chỉ được sử dụng duy nhất cho việc xử lý đếm lùi các timer phần mềm. ọi xử lý hoặc tính toán phức tạp phải được đưa vào vòng lặp while trong hàm chính để tối ưu hóa sự phức tạp cho hàm phục vụ ngắt

Ý tưởng: Mở rộng timer phần mềm thành mảng để nhiều tác vụ đồng thời. Cấp phát timer 0 cho đèn 4 LED 7 đoạn, timer 1 cho DOT và timer 2 cho đèn LED. Tốc độ quét cần được cài đặt ở mức phù hợp.

```
    updateClockBuffer();
  setTimer(0, 250);  //  LED (250ms)
  setTimer(1, 500);  // DOT (500ms)
  setTimer(2, 1000); //  Clock & LED_RED (1000ms)
  HAL_TIM_Base_Start_IT(&htim2);
/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
  while (1)
  {
  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */

      if(timer_flag[0] == 1){
          HAL_GPIO_WritePin(GPIOA,GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|
 GPIO_PIN_9, GPIO_PIN_SET);
          update7SEG(index_led);
          setTimer(0, 250);
      }
      if(timer_flag[1] == 1){
          HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
          setTimer(1, 500);
      }
      if(timer_flag[2] == 1){
          HAL_GPIO_TogglePin(LED_RED_GPIO_Port, LED_RED_Pin);

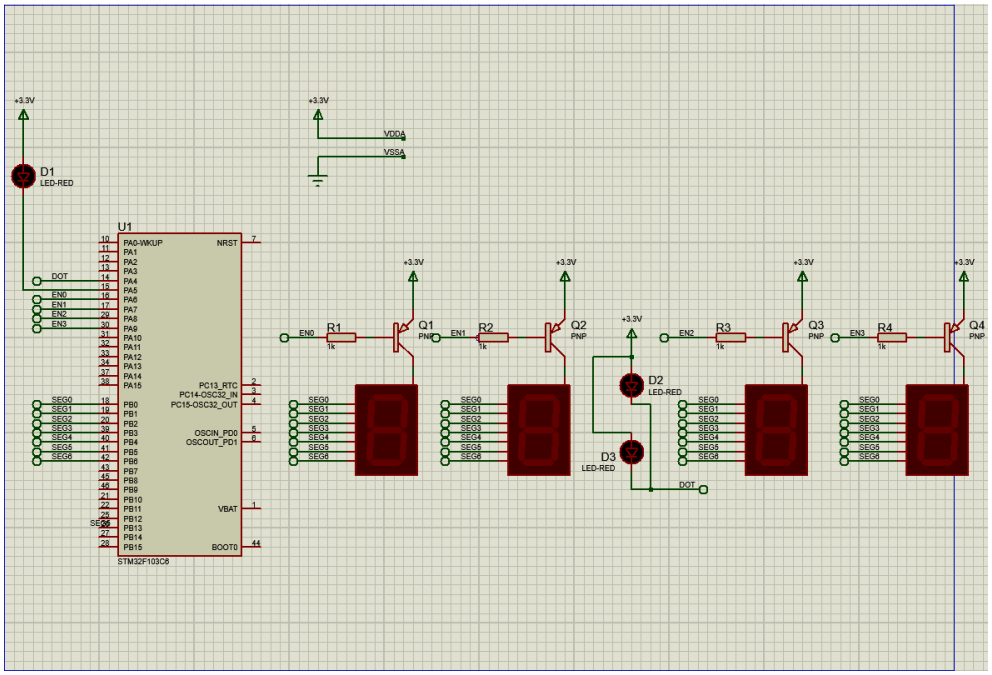
          second++;
          if(second >= 60) {
              second = 0;
              minute++;
          }
          if(minute >= 60) {
```

```
                minute = 0;
                hour++;
            }
            if(hour >= 24) {
                hour = 0;
            }
            updateClockBuffer();
            setTimer(2, 1000);
        }
    }
  /* USER CODE END 3 */


  void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
  if(htim->Instance == TIM2){
    timer_run();
  }

}
```

Listing 11: Phần code hiện thực



Hình 2: *Mô phỏng bài 8*

11 Exercise 6 - Lab 2

Yêu cầu: Mục tiêu chính là giảm thiểu độ phức tạp bên trong hàm ngắt timer để tránh lỗi ngắt lồng nhau gây treo hệ thống. Tạo ra một software timer đếm lùi sau mỗi lần

ngắt timer xảy ra mỗi 10ms. Sử dụng timer phần mềm này để thay thế và loại bỏ hàm `HAL_Delay(1000)` trong hàm `main()`, nhằm tạo ra một hệ thống xử lý trễ không đồng bộ.

Ý tưởng: Khai báo các biến toàn cục để quản lý timer phần mềm gồm: `timer_counter`, `timer_flag` và `TIMER_CYCLE`. Hiện thực hàm `setTimer(int duration)` để nạp giá trị đếm bằng cách lấy thời gian mong muốn chia cho chu kỳ. Hàm `timer_run()` để lùi biến `timer_counter` lớn hơn 0, khi biến đếm về 0 thì cờ bật lên 1.

```
int timer_counter = 0;
int timer_flag = 0;
int TIMER_CYCLE = 10;

void setTimer(int duration){
    timer_counter = duration / TIMER_CYCLE;
    timer_flag = 0;
}

void timer_run(){
    if(timer_counter > 0){
        timer_counter--;
        if(timer_counter == 0) timer_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if(htim->Instance == TIM2){
        timer_run();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|
   GPIO_PIN_9, GPIO_PIN_SET);
        update7SEG(index_led);

        if(second_counter == 2 || second_counter == 4) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        }

        second++;
        if(second >= 60) {
            second = 0;
            minute++;
        }

        if(minute >= 60) {
            minute = 0;
            hour++;
```

```
        }

        if(hour >= 24) {
            hour = 0;
        }

        updateClockBuffer();
    }
}
```

Listing 12: Phần code hiện thực

- **Report 1:** Nếu xóa dòng `setTimer(1000);` trước vòng lặp while, biến `timer_counter` ban đầu sẽ bằng 0. Cờ sẽ không bao giờ được bật lên 1, dẫn đến LED không bao giờ nhấp nháy.
- **Report 2:** Nếu đổi thành `setTimer(1);`, thời gian đếm lùi sẽ là $1/10 = 0$. LED sẽ không nhấp nháy.
- **Report 3:** Nếu đổi thành `setTimer(10);`, LED sẽ nhấp nháy mỗi 10ms. LED sẽ nhấp nháy nhanh, chúng ta vẫn sẽ không nhìn ra được sự nhấp nháy.

12 Exercise 6 - Lab 2

Yêu cầu: Mục tiêu chính là giảm thiểu độ phức tạp bên trong hàm ngắt timer để tránh lỗi ngắt lồng nhau gây treo hệ thống. Tạo ra một software timer đếm lùi sau mỗi lần ngắt timer xảy ra mỗi 10ms. Sử dụng timer phần mềm này để thay thế và loại bỏ hàm `HAL_Delay(1000)` trong hàm `main()`, nhằm tạo ra một hệ thống xử lý trễ không đồng bộ.

Ý tưởng: Khai báo các biến toàn cục để quản lý timer phần mềm gồm: `timer_counter`, `timer_flag` và `TIMER_CYCLE`. Hiện thực hàm `setTimer(int duration)` để nạp giá trị đếm bằng cách lấy thời gian mong muốn chia cho chu kỳ. Hàm `timer_run()` để lùi biến `timer_counter` lớn hơn 0, khi biến đếm về 0 thì cờ bật lên 1.

```
int timer_counter = 0;
int timer_flag = 0;
int TIMER_CYCLE = 10;

void setTimer(int duration){
    timer_counter = duration / TIMER_CYCLE;
    timer_flag = 0;
}

```

```c
void timer_run(){
    if(timer_counter > 0){
        timer_counter--;
        if(timer_counter == 0) timer_flag = 1;
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){
    if(htim->Instance == TIM2){
        timer_run();
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_6|GPIO_PIN_7|GPIO_PIN_8|
   GPIO_PIN_9, GPIO_PIN_SET);
        update7SEG(index_led);

        if(second_counter == 2 || second_counter == 4) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_4);
        }

        second++;
        if(second >= 60) {
            second = 0;
            minute++;
        }

        if(minute >= 60) {
            minute = 0;
            hour++;
        }

        if(hour >= 24) {
            hour = 0;
        }

        updateClockBuffer();
    }
}
```

Listing 13: Phần code hiện thực

- **Report 1:** Nếu xóa dòng `setTimer(1000);` trước vòng lặp while, biến `timer_counter` ban đầu sẽ bằng 0. Cờ sẽ không bao giờ được bật lên 1, dẫn đến LED không bao giờ nhấp nháy.
- **Report 2:** Nếu đổi thành `setTimer(1);`, thời gian đếm lùi sẽ là $1/10 = 0$. LED sẽ không nhấp nháy.

- **Report 3:** Nếu đổi thành `setTimer(10);`, LED sẽ nhấp nháy mỗi 10ms. LED sẽ nhấp nháy nhanh, chúng ta vẫn sẽ không nhìn ra được sự nhấp nháy.

13 Lab 3

13.1 Exercise 1 - Lab 3

Yêu cầu: Thiết kế và phác thảo Máy trạng thái hữu hạn (FSM) cho hệ thống đèn giao thông tại ngã tư đáp ứng các yêu cầu sau:

- Điều khiển 12 đèn LED (4 Đỏ, 4 Vàng, 4 Xanh) và 4 LED 7 đoạn (2 LED hiển thị giá trị thời gian, 2 LED hiển thị chế độ hoạt động).
- Sử dụng 3 nút nhấn:
  - **Button 1**: Chuyển đổi giữa các chế độ (Mode 1 → Mode 2 → Mode 3 → Mode 4 → Mode 1).
  - **Button 2**: Tăng giá trị thời gian cài đặt (1 → 99).
  - **Button 3**: Lưu (Set) giá trị thời gian vừa cài đặt.

Mô tả Tín hiệu và Trạng thái FSM Tín hiệu Input / Output

- **Inputs**: `is_button_pressed(0)` (Button 1), `is_button_pressed(1)` (Button 2), `is_button_pressed(2)` (Button 3).
- **Outputs**: Trạng thái hiển thị 12 LED giao thông, mã chế độ (`MODE_01` đến `MODE_04`) và giá trị thời gian đếm ngược/cài đặt trên 4 LED 7 đoạn.

Chi tiết các trạng thái

1. **MODE 1 - Chế độ tự động:**
   - *Hành vi*: Hệ thống đèn giao thông chuyển màu tự động theo chu kỳ (Đỏ - Xanh - Vàng). LED 7 đoạn hiển thị thời gian đếm ngược của mỗi làn đường.
   - *Chuyển trạng thái*: Khi bấm `Button 1`, FSM chuyển sang **MODE 2**.
2. **MODE 2 - Cài đặt thời gian đèn Đỏ:**
   - *Hành vi*: Tất cả đèn Đỏ nhấp nháy ở tần số 2Hz. 2 LED 7 đoạn hiển thị mã Mode `02`, 2 LED còn lại hiển thị giá trị thời gian đèn Đỏ đang chỉnh.
   - *Thao tác*: Bấm `Button 2` để tăng thời gian (1 → 99), bấm `Button 3` để lưu giá trị.
   - *Chuyển trạng thái*: Khi bấm `Button 1`, FSM chuyển sang **MODE 3**.
3. **MODE 3 - Cài đặt thời gian đèn Vàng:**

- *Hành vi*: Tất cả đèn Vàng nhấp nháy ở tần số 2Hz. 2 LED 7 đoạn hiển thị mã Mode `03`, 2 LED còn lại hiển thị giá trị thời gian đèn Vàng.
- *Thao tác*: Bấm `Button 2` để tăng thời gian, bấm `Button 3` để lưu giá trị.
- *Chuyển trạng thái*: Khi bấm `Button 1`, FSM chuyển sang **MODE 4**.

4. **MODE 4 - Cài đặt thời gian đèn Xanh:**

- *Hành vi*: Tất cả đèn Xanh nhấp nháy ở tần số 2Hz. 2 LED 7 đoạn hiển thị mã Mode `04`, 2 LED còn lại hiển thị giá trị thời gian đèn Xanh.
- *Thao tác*: Bấm `Button 2` để tăng thời gian, bấm `Button 3` để lưu giá trị.
- *Chuyển trạng thái*: Khi bấm `Button 1`, FSM quay trở lại **MODE 1**.

Bảng chuyển trạng thái và sơ đồ FSM

Bảng 1: *Bảng chuyển đổi trạng thái của FSM theo sự kiện nút nhấn*

Hiện tại	Đầu vào	Kế tiếp	Hành động
`MODE_1`	Press Button 1	`MODE_2`	Bật nháy LED Đỏ 2Hz, hiển thị Mode 02
`MODE_2`	Press Button 2	`MODE_2`	Tăng giá trị tạm thời `temp_red_time`
`MODE_2`	Press Button 3	`MODE_2`	Lưu `red_time = temp_red_time`
`MODE_2`	Press Button 1	`MODE_3`	Bật nháy LED Vàng 2Hz, hiển thị Mode 03
`MODE_3`	Press Button 2	`MODE_3`	Tăng giá trị tạm thời `temp_amber_time`
`MODE_3`	Press Button 3	`MODE_3`	Lưu `amber_time = temp_amber_time`
`MODE_3`	Press Button 1	`MODE_4`	Bật nháy LED Xanh 2Hz, hiển thị Mode 04
`MODE_4`	Press Button 2	`MODE_4`	Tăng giá trị tạm thời `temp_green_time`
`MODE_4`	Press Button 3	`MODE_4`	Lưu `green_time = temp_green_time`
`MODE_4`	Press Button 1	`MODE_1`	Cập nhật chu kỳ mới và chạy tự động

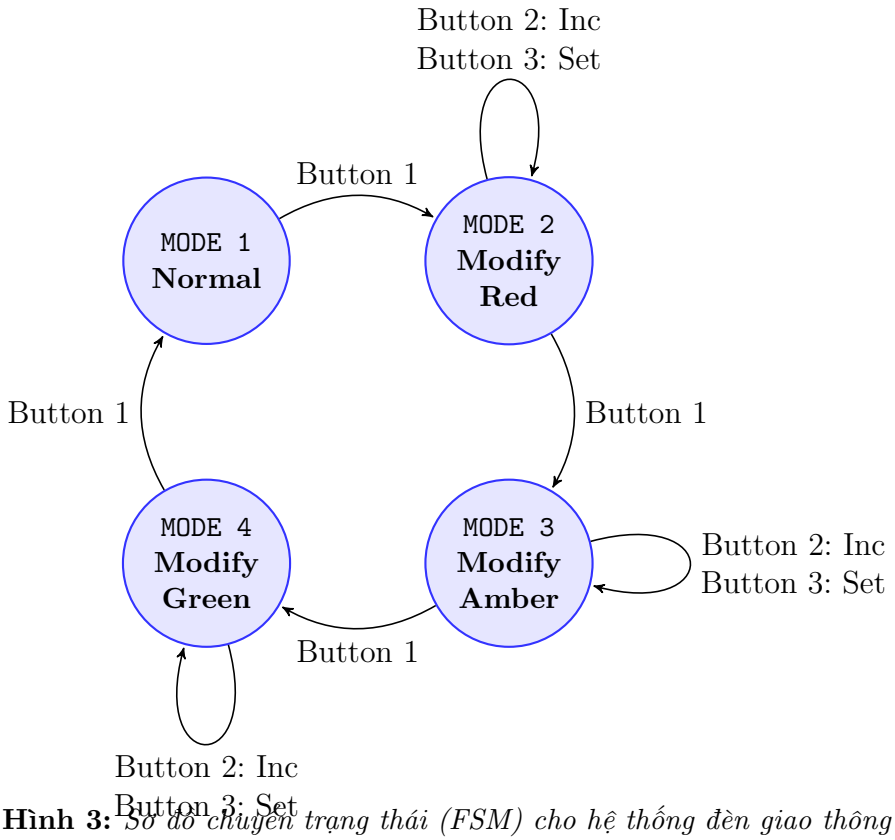
13.2 Exercise 3 - Lab 3

Yêu cầu: Tạo dự án STM32CubeIDE/STM32CubeMX, thiết lập cấu hình các chân GPIO cho 3 nút nhấn, 12 LED giao thông, 4 LED 7 đoạn và cấu hình ngắt Timer 2 với chu kỳ 10ms phục vụ quét nút nhấn và quét hiển thị[cite: 14, 21, 22].

Cấu hình Pinout (File .ioc) Các chân ngoại vi trên vi điều khiển STM32F103C6 được thiết lập chi tiết trong Bảng 2.

Cấu hình Timer 2 Ngắt 10ms Để tạo chu kỳ ngắt chính xác 10ms với tần số xung nhịp hệ thống 8MHz[cite: 14, 22]:

- **Clock Source**: Internal Clock.
- **Prescaler (PSC)**: 7999 (đưa tần số định thời về $8\text{MHz}/(7999+1) = 1\text{kHz}$, tương ứng 1ms/tick).



Hình 3: *Sơ đồ chuyển trạng thái (FSM) cho hệ thống đèn giao thông*

Bảng 2: *Bảng phân hoạch chân I/O hệ thống trong STM32CubeMX*

Chân	Chế độ (Mode)	Pull-up / Speed	User Label	Chức năng
PB0	GPIO_Input	Pull-up	BUTTON_1	Nút chọn Mode (Mode 1
PB1	GPIO_Input	Pull-up	BUTTON_2	Nút tăng thời gian (1 →
PB2	GPIO_Input	Pull-up	BUTTON_3	Nút lưu giá trị (Set)[cite
PA0 – PA11	GPIO_Output	Output Push-Pull	RED_1 – GRN_4	12 LED giao thông[cite:
PB3 – PB9	GPIO_Output	Output Push-Pull	SEG_A – SEG_G	7 thanh dữ liệu LED 7 đ
PB10 – PB13	GPIO_Output	Output Push-Pull	EN_0 – EN_3	4 chân quét chọn LED 7

- **Counter Period (ARR)**: 9 (đếm 10 ticks từ 0 → 9, tương ứng khoảng thời gian ngắt 10ms).
- **NVIC Settings**: Kích hoạt ngắt toàn cục `TIM2 global interrupt` (Tick chọn `Enabled`).

Kết quả khởi tạo Sau khi thực hiện *Generate Code*, STM32CubeMX tự động sinh các hàm khởi tạo ngoại vi trong `main.c`[cite: 20]:

- `MX_GPIO_Init()`: Cấu hình các chân điều khiển LED là Output và các chân nút nhấn là Input với điện trở kéo lên (*Pull-up*)[cite: 10, 20].
- `MX_TIM2_Init()`: Khởi tạo các thông số thanh ghi cho Timer 2[cite: 20].

Để khởi động Timer 2 ở chế độ ngắt trong chương trình chính, ta thêm lệnh sau trước vòng lặp `while(1)`:

```
/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_TIM2_Init();

/* Start Timer 2 in Interrupt mode */
HAL_TIM_Base_Start_IT(&htim2);

while (1)
{
    /* Super loop */
}
```

Listing 14: Khởi động ngắt Timer 2 trong main.c

13.3 Exercise 4 - Lab 3

Yêu cầu: Trong các ứng dụng nhúng, chu kỳ ngắt Timer phần cứng có thể thay đổi tùy theo yêu cầu hệ thống. Nếu các giá trị đếm thời gian được thiết lập cứng, toàn bộ các tác vụ đếm chu kỳ (như nháy LED 2Hz, đếm ngược 1s) sẽ bị sai lệch nghiêm trọng.

Mục tiêu của Bài tập 4 là xây dựng một lớp trừu tượng hóa Software Timer dựa trên hằng số `TIMER_CYCLE`. Khi chu kỳ ngắt của Timer phần cứng thay đổi, người lập trình chỉ cần cập nhật duy nhất hằng số `TIMER_CYCLE` mà không cần chỉnh sửa bất kỳ dòng mã nguồn nào ở các tầng ứng dụng FSM bên trên.

Thiết kế Software Timer dùng Mảng và `TIMER_CYCLE`

Cấu trúc dữ liệu Để quản lý đồng thời nhiều tác vụ định thời độc lập (như quét LED, nháy LED 2Hz, đếm ngược 1s), hệ thống sử dụng mảng biến toàn cục quản lý theo chỉ số `index`:

- `TIMER_CYCLE`: Chu kỳ ngắt phần cứng tính bằng miligiây (ms). Với cấu hình TIM2 ngắt 10ms, ta khai báo `#define TIMER_CYCLE 10`.
- `timer_counter[4]`: Mảng lưu số tick còn lại của từng Software Timer.
- `timer_flag[4]`: Mảng cờ báo hiệu khi một Software Timer tương ứng đếm xong ($0 \rightarrow 1$).

Công thức quy đổi tự động Hàm `setTimer(index, duration)` tiếp nhận khoảng thời gian mong muốn `duration` (ms) và tự động quy đổi ra số lần ngắt (tick) tương ứng theo công thức:

$$\text{timer_counter}[index] = \frac{\text{duration}}{\text{TIMER_CYCLE}}$$

Mã nguồn Cài đặt (`timer.h` và `timer.c`)

```c
#ifndef INC_TIMER_H_
#define INC_TIMER_H_

void setTimer(int index, int duration);
void timer_run(void);


#endif /* INC_TIMER_H_ */
```

Listing 15: File timer.h

```c
#include "main.h"
#include "timer.h"
#include "input_reading.h"

extern int time_buffer[4];
extern int timer_counter[4];
extern int timer_flag[4];
extern int TIMER_CYCLE;

void setTimer(int index, int duration){
    timer_counter[index] = duration / TIMER_CYCLE;
    timer_flag[index] = 0;
}

void timer_run(){
    for(int i = 0; i < 4; i++){
        if(timer_counter[i] > 0){
            timer_counter[i]--;
            if(timer_counter[i] == 0) timer_flag[i] = 1;
        }
```

```
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM2) {
        button_reading();
        timer_run();
    }
}
```

Listing 16: Khởi động ngắt Timer 2 trong main.c

Đánh giá và Kết quả

- **Độ linh hoạt cao**: Khi ngắt `TIM2` cài đặt 10ms (`TIMER_CYCLE = 10`), gọi `setTimer(0, 500)` phục vụ nháy LED 2Hz sẽ tự động tính số tick là $500/10 = 50$ ticks.
- **Dễ dàng bảo trì**: Nếu sau này trong STM32CubeMX chu kỳ ngắt `TIM2` được đổi thành 1ms, ta chỉ cần đổi `#define TIMER_CYCLE 1`. Lệnh `setTimer(0, 500)` sẽ tự động tính ra $500/1 = 500$ ticks, giữ nguyên chính xác khoảng thời gian 500ms mà không làm lệch tần số hoạt động của ứng dụng.

13.4 Exercise 5 - Lab 3: Button Debouncing Abstraction Layer

Yêu cầu: Trong thực tế, các nút nhấn cơ học khi thao tác luôn xảy ra hiện tượng dội tín hiệu trong khoảng 10ms $\rightarrow$ 20ms đầu tiên, làm vi điều khiển nhận diện nhầm một lần nhấn thành nhiều lần nhấn liên tục.

Để giải quyết triệt để hiện tượng này, bài tập 5 triển khai thuật toán lấy mẫu 3 lần liên tiếp bên trong ngắt Timer 2 với chu kỳ 10ms:

1. Cập nhật 3 thanh ghi lấy mẫu: `KeyReg0`, `KeyReg1`, `KeyReg2`.
2. Chỉ xác nhận trạng thái nút nhấn khi cả 3 thanh ghi thu được cùng một giá trị ổn định trong ít nhất 30ms (3×10ms).
3. Sử dụng thanh ghi `KeyReg3` để lưu trạng thái ổn định và xác định thời điểm bắt đầu nhấn (*Falling edge detection*) để kích hoạt cờ `button_flag`.

Mã nguồn Cài đặt

```
#ifndef INC_INPUT_READING_H_
#define INC_INPUT_READING_H_

#include"main.h"
```

```
#include "timer.h"

void init_buffer();
void button_reading(void);
unsigned char is_button_pressed(unsigned char index);
unsigned char is_button_pressed_1s(unsigned char index);
unsigned char is_button_held(unsigned char index);
void reset_flagForButtonHold(unsigned char index);

#endif /* INC_INPUT_READING_H_ */
```

Listing 17: File input reading.h

```
static GPIO_PinState buttonBuffer[NO_OF_BUTTONS] = {BUTTON_IS_RELEASED,
    BUTTON_IS_RELEASED, BUTTON_IS_RELEASED};
static GPIO_PinState debounceButtonBuffer1[NO_OF_BUTTONS] = {
    BUTTON_IS_RELEASED, BUTTON_IS_RELEASED, BUTTON_IS_RELEASED};
static GPIO_PinState debounceButtonBuffer2[NO_OF_BUTTONS] = {
    BUTTON_IS_RELEASED, BUTTON_IS_RELEASED, BUTTON_IS_RELEASED};
static GPIO_PinState debounceButtonBuffer3[NO_OF_BUTTONS] = {
    BUTTON_IS_RELEASED, BUTTON_IS_RELEASED, BUTTON_IS_RELEASED};

static uint8_t flagForButtonPress1s[NO_OF_BUTTONS] = {0, 0, 0};
static uint16_t counterForButtonPress1s[NO_OF_BUTTONS] = {0, 0, 0};
static uint8_t flagForButtonHold[NO_OF_BUTTONS] = {0, 0, 0};
static uint16_t counterForButtonHold[NO_OF_BUTTONS] = {0, 0, 0};
```

Listing 18: Cài đặt lọc nhiễu dội phím trong file input reading.c

```
void button_reading(void) {
    for (int i = 0; i < NO_OF_BUTTONS; i++) {
        debounceButtonBuffer3[i] = debounceButtonBuffer2[i];
        debounceButtonBuffer2[i] = debounceButtonBuffer1[i];


        debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(button_ports[i],
    button_pins[i]);

        if (debounceButtonBuffer1[i] == debounceButtonBuffer2[i] &&
    debounceButtonBuffer3[i] == debounceButtonBuffer2[i])
            buttonBuffer[i] = debounceButtonBuffer1[i];

        if (buttonBuffer[i] == BUTTON_IS_PRESSED) {
            if (counterForButtonPress1s[i] <
    DURATION_FOR_AUTO_INCREASING) {
                counterForButtonPress1s[i]++;
```

```
            } else {
                flagForButtonPress1s[i] = 1;
                if (counterForButtonHold[i] < DURATION_FOR_HOLD) {
                    counterForButtonHold[i]++;
                    if (counterForButtonHold[i] >= DURATION_FOR_HOLD) {
                        counterForButtonHold[i] = 0;
                        flagForButtonHold[i] = 1;
                    }
                }
            }
        } else {
            counterForButtonPress1s[i] = 0;
            flagForButtonPress1s[i] = 0;
            counterForButtonHold[i] = 0;
            flagForButtonHold[i] = 0;
        }
    }
}
unsigned char is_button_pressed(uint8_t index){
    if(index >= NO_OF_BUTTONS) return 0;
    return (buttonBuffer[index] == BUTTON_IS_PRESSED);
}

unsigned char is_button_pressed_1s(unsigned char index){
    if(index >= NO_OF_BUTTONS) return 0xff;
    return (flagForButtonPress1s[index] == 1);
}

unsigned char is_button_held(unsigned char index) {
    if (index >= NO_OF_BUTTONS) return 0;
    return (flagForButtonHold[index] == 1);
}
```

Listing 19: Hàm xử lý phím trong file input reading.c

14 Exercise 5 - Lab 3: Button Debouncing Abstraction Layer

Yêu cầu: Trong thực tế, các nút nhấn cơ học khi thao tác luôn xảy ra hiện tượng dội tín hiệu trong khoảng 10ms → 20ms đầu tiên, làm vi điều khiển nhận diện nhầm một lần nhấn thành nhiều lần nhấn liên tục.

Để giải quyết triệt để hiện tượng này, bài tập 5 triển khai thuật toán lấy mẫu 3 lần

liên tiếp bên trong ngắt Timer 2 với chu kỳ 10ms:

1. Cập nhật 3 thanh ghi lấy mẫu: `KeyReg0`, `KeyReg1`, `KeyReg2`.
2. Chỉ xác nhận trạng thái nút nhấn khi cả 3 thanh ghi thu được cùng một giá trị ổn định trong ít nhất 30ms (3×10ms).
3. Sử dụng thanh ghi `KeyReg3` để lưu trạng thái ổn định và xác định thời điểm bắt đầu nhấn (*Falling edge detection*) để kích hoạt cờ `button_flag`.

14.1 Mã nguồn Cài đặt

```
#ifndef INC_INPUT_READING_H_
#define INC_INPUT_READING_H_

#include"main.h"
#include "timer.h"

void init_buffer();
void button_reading(void);
unsigned char is_button_pressed(unsigned char index);
unsigned char is_button_pressed_1s(unsigned char index);
unsigned char is_button_held(unsigned char index);
void reset_flagForButtonHold(unsigned char index);

#endif /* INC_INPUT_READING_H_ */
```

Listing 20: File input reading.h

```
static GPIO_PinState buttonBuffer[NO_OF_BUTTONS] = {BUTTON_IS_RELEASED,
    BUTTON_IS_RELEASED, BUTTON_IS_RELEASED};
static GPIO_PinState debounceButtonBuffer1[NO_OF_BUTTONS] = {
    BUTTON_IS_RELEASED, BUTTON_IS_RELEASED, BUTTON_IS_RELEASED};
static GPIO_PinState debounceButtonBuffer2[NO_OF_BUTTONS] = {
    BUTTON_IS_RELEASED, BUTTON_IS_RELEASED, BUTTON_IS_RELEASED};
static GPIO_PinState debounceButtonBuffer3[NO_OF_BUTTONS] = {
    BUTTON_IS_RELEASED, BUTTON_IS_RELEASED, BUTTON_IS_RELEASED};

static uint8_t flagForButtonPress1s[NO_OF_BUTTONS] = {0, 0, 0};
static uint16_t counterForButtonPress1s[NO_OF_BUTTONS] = {0, 0, 0};
static uint8_t flagForButtonHold[NO_OF_BUTTONS] = {0, 0, 0};
static uint16_t counterForButtonHold[NO_OF_BUTTONS] = {0, 0, 0};
```

Listing 21: Cài đặt lọc nhiễu dội phím trong file input reading.c

```c
void button_reading(void) {
    for (int i = 0; i < NO_OF_BUTTONS; i++) {
        debounceButtonBuffer3[i] = debounceButtonBuffer2[i];
        debounceButtonBuffer2[i] = debounceButtonBuffer1[i];


        debounceButtonBuffer1[i] = HAL_GPIO_ReadPin(button_ports[i], button_pins[i]);

        if (debounceButtonBuffer1[i] == debounceButtonBuffer2[i] && debounceButtonBuffer3[i] == debounceButtonBuffer2[i])
            buttonBuffer[i] = debounceButtonBuffer1[i];

        if (buttonBuffer[i] == BUTTON_IS_PRESSED) {
            if (counterForButtonPress1s[i] < DURATION_FOR_AUTO_INCREASING) {
                counterForButtonPress1s[i]++;
            } else {
                flagForButtonPress1s[i] = 1;
                if (counterForButtonHold[i] < DURATION_FOR_HOLD) {
                    counterForButtonHold[i]++;
                    if (counterForButtonHold[i] >= DURATION_FOR_HOLD) {
                        counterForButtonHold[i] = 0;
                        flagForButtonHold[i] = 1;
                    }
                }
            }
        } else {
            counterForButtonPress1s[i] = 0;
            flagForButtonPress1s[i] = 0;
            counterForButtonHold[i] = 0;
            flagForButtonHold[i] = 0;
        }
    }
}
unsigned char is_button_pressed(uint8_t index){
    if(index >= NO_OF_BUTTONS) return 0;
    return (buttonBuffer[index] == BUTTON_IS_PRESSED);
}

unsigned char is_button_pressed_1s(unsigned char index){
    if(index >= NO_OF_BUTTONS) return 0xff;
    return (flagForButtonPress1s[index] == 1);
}

```

```
43 unsigned char is_button_held(unsigned char index) {
44     if (index >= NO_OF_BUTTONS) return 0;
45     return (flagForButtonHold[index] == 1);
46 }
```

Listing 22: Hàm xử lý phím trong file input reading.c